Lezione 11 - Gestire gli Errori

Introduzione

Fino a questo momento abbiamo studiato i concetti fondamentali della programmazione orientata agli oggetti in Java: definizione di classi, attributi e metodi, costruttori, incapsulamento, overriding e meccanismo di ereditarietà. Abbiamo visto come modellare oggetti e come organizzare il codice in modo strutturato, ma abbiamo sempre dato per scontato che il programma venisse eseguito in condizioni ideali, senza errori durante l'esecuzione.

Nella realtà, però, un programma può trovarsi in situazioni impreviste: input non valido, accesso a un indice fuori dai limiti di un array, divisione per zero, file non trovati, riferimenti nulli. Questi problemi non vengono rilevati in fase di compilazione, ma emergono durante l'esecuzione del programma, generando errori a runtime.

Per affrontare in modo controllato queste situazioni, Java mette a disposizione il **meccanismo delle eccezioni**, che consente di:

- **segnalare un errore nel momento in cui si verifica (`throw`);**
- **dichiarare che un metodo può generare un'eccezione (`throws`);**
- **intercettare e gestire l'errore senza interrompere bruscamente il programma (`try-catch`);**
- **garantire l'esecuzione di codice critico tramite il blocco `finally`.**

Introdurremo inoltre la distinzione tra **eccezioni checked e unchecked**, analizzeremo il meccanismo di **propagazione lungo la catena delle chiamate**, vedremo come creare **eccezioni personalizzate** e quali vincoli si applicano in caso di **overriding** dei metodi.

In questo modo completiamo il modello di esecuzione di un programma Java, passando dalla semplice definizione di strutture e comportamenti alla gestione strutturata delle situazioni di errore.

Eccezioni e gestione degli errori in Java

In un programma reale è inevitabile che si verifichino situazioni anomale: input non validi, dispositivi non disponibili, condizioni impreviste durante l'esecuzione.

Per questo motivo Java mette a disposizione un **meccanismo strutturato di gestione degli errori:**

- **consente di trasferire il flusso di esecuzione dal punto in cui si è verificato il problema a un gestore degli errori (*exception handler*), in grado di affrontare la situazione in modo controllato**.

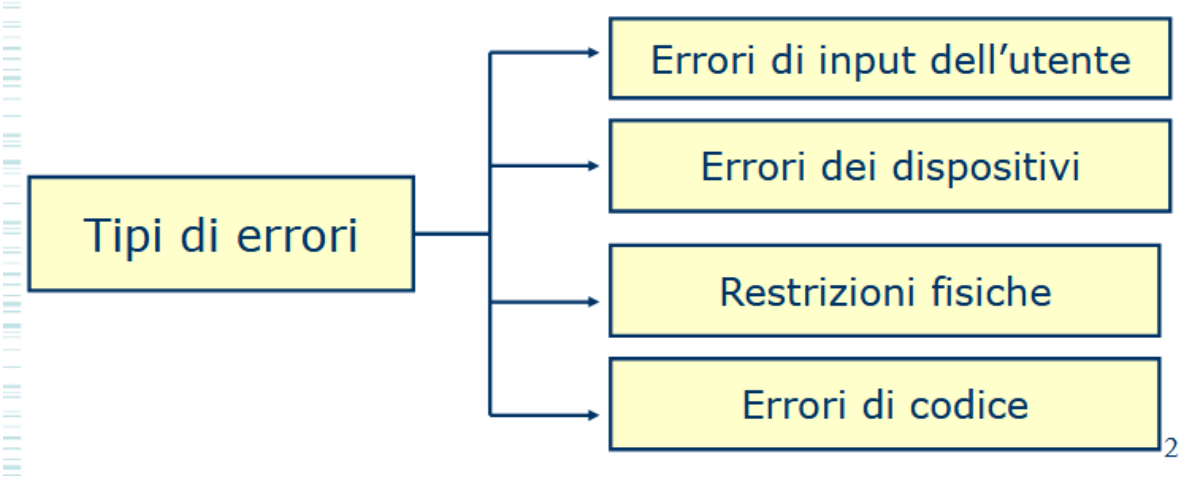
L'obiettivo non è solo intercettare l'errore, ma:

- **evitare l'interruzione improvvisa del programma,**
- **mantenere la coerenza dello stato dell'applicazione,**
- **fornire informazioni utili per la diagnosi del problema**.

Tipi di errori

Gli errori che possono verificarsi in un'applicazione possono essere classificati in diverse categorie:

- **Errori di input dell'utente**
  Ad esempio: inserimento di un valore numerico non valido, formato errato di una data, dati mancanti.
- **Errori dei dispositivi**
  Problemi legati a risorse esterne come file non trovati, connessione di rete assente, stampante non disponibile.
- **Errori di codice**
  Errori logici o di programmazione, come accesso a un indice fuori dai limiti di un array, divisione per zero, utilizzo di un riferimento `null`.



Gestione degli Errori

Nella programmazione tradizionale, **la gestione degli errori avveniva tramite una tecnica piuttosto semplice**:

- **quando un metodo rilevava un problema, restituiva un codice di errore speciale che il metodo chiamante doveva controllare e interpretare.** Questo approccio, tuttavia, rendeva il codice meno leggibile e più difficile da mantenere, poiché obbligava a verificare costantemente i valori di ritorno.

Java introduce un meccanismo più strutturato ed espressivo: invece di restituire un valore "anomalo", un metodo può terminare la propria esecuzione attraverso un **percorso alternativo**, lanciando un oggetto che rappresenta l'errore verificatosi.
Questo oggetto appartiene alla gerarchia delle **eccezioni**.

Le eccezioni sono infatti oggetti derivati dalla classe `Exception` e modellano situazioni anomale che si verificano durante l'esecuzione del programma.

Quando si verifica un'eccezione:

- **il flusso normale di esecuzione viene interrotto;**
- **il sistema ricerca un gestore dell'errore (exception handler) in grado di trattare la situazione;**
- **se nessun gestore è presente, il programma termina mostrando informazioni sull'errore.**

Esempio concettuale

Si consideri un metodo che effettua la divisione tra due interi:

Java

```java
int risultato = a / b;
```

Se il divisore `b` vale `0`, si verifica un errore aritmetico. In questo caso Java genera automaticamente un'eccezione, ad esempio:

Java

```
java.lang.ArithmeticException: / by zero
```

L'esecuzione viene quindi interrotta e il controllo passa al meccanismo di gestione delle eccezioni.

Error ed Exception

In Java, tutto ciò che può essere “lanciato” (cioè segnalato come evento anomalo durante l’esecuzione) deriva dalla classe fondamentale:

```java
Throwable
```

`Throwable` rappresenta la radice della gerarchia degli errori e delle eccezioni e si suddivide in **due grandi famiglie**:
`Error` ed `Exception`.
Questa distinzione è fondamentale perché identifica la natura e la gravità del problema che si è verificato.

La gerarchia `Error`

La classe `Error`:

- **rappresenta problemi gravi legati al funzionamento interno della JVM o all’ambiente di esecuzione.**

Si tratta generalmente di:

- **errori interni del sistema,**
- **esaurimento delle risorse (ad esempio memoria insufficiente),**
- **malfunzionamenti critici dell’ambiente runtime.**

Un esempio tipico è:

```java
OutOfMemoryError
```

Questi errori non sono normalmente gestibili dall’applicazione, poiché indicano condizioni che compromettono il corretto funzionamento della JVM stessa.
In altre parole, quando si verifica un `Error`, il problema non riguarda la logica del programma, ma l’infrastruttura su cui esso sta girando.

La gerarchia `Exception`

La classe `Exception`, invece:

- **La classe `Exception`, invece, rappresenta situazioni anomale che possono verificarsi durante l’esecuzione di un programma ma che, a**

differenza degli `Error`, possono essere intercettate e gestite.

Rientrano in questa categoria:

- errori di input,
- accesso a file inesistenti,
- divisione per zero,
- accesso a riferimenti null,
- e molte altre condizioni impreviste ma controllabili.

A differenza degli `Error`, le `Exception` fanno parte della normale progettazione di un'applicazione robusta:

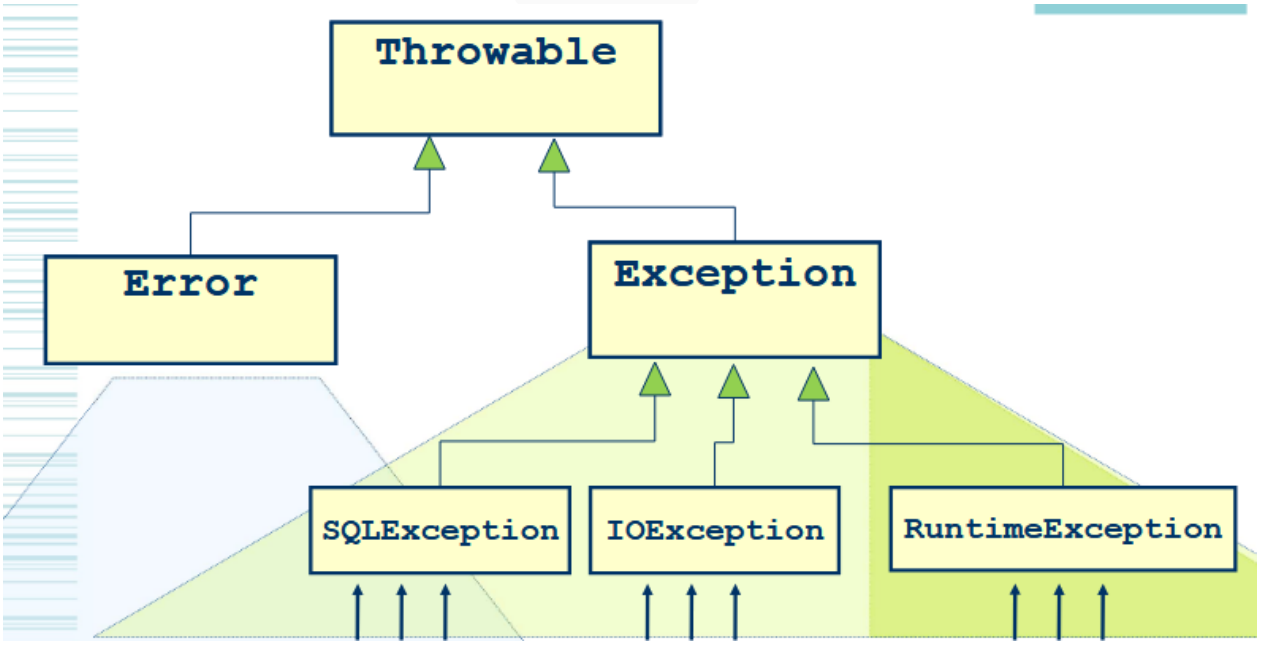
- **il programmatore può prevederle e gestirle attraverso meccanismi specifici del linguaggio, garantendo così maggiore stabilità e controllo del flusso di esecuzione**

Struttura della gerarchia

Sia `Error` sia `Exception` derivano da `Throwable` e riportano nel nome l'indicazione del problema verificatosi (ad esempio `ArithmeticException`, `NullPointerException`, `IOException`).

Questa organizzazione gerarchica consente a Java di distinguere chiaramente tra:

- problemi **critici di sistema** (`Error`),
- anomalie **applicative gestibili** (`Exception`).



Analizziamo la struttura rappresentata nell'immagine.

Il concetto fondamentale è che **tutte le classi che possono essere lanciate tramite `throw` derivano da `Throwable`**.
`Throwable` costituisce quindi la radice dell'intera gerarchia delle eccezioni in Java.

Da questa classe derivano direttamente due rami principali

```
Throwable
  ├── Error
  └── Exception
```

Si tratta di una relazione di ereditarietà (*is-a*), per cui:

- `Error` è un `Throwable`
- `Exception` è un `Throwable`

Proseguendo lungo l'albero gerarchico si incontrano le sottoclassi dei due rami principali.

1. Ramo `Error`

Nell'immagine non sono riportate le sottoclassi di `Error`, ma anche questa classe possiede diverse specializzazioni che rappresentano errori gravi della JVM o dell'ambiente di esecuzione, ad esempio:

```
Error
  ├── OutOfMemoryError
  ├── StackOverflowError
  └── VirtualMachineError
```

Ogni sottoclasse rappresenta una forma più specifica di errore, mantenendo comunque la relazione di ereditarietà:

- `OutOfMemoryError` è un `Error`
- `Error` è un `Throwable`

2. Ramo `Exception`

Il ramo `Exception` è più articolato e contiene numerose sottoclassi.
Anche in questo caso la relazione *is-a* rimane valida lungo tutta la catena gerarchica. Ad esempio:

- `FileNotFoundException` è una `IOException`
- `IOException` è una `Exception`
- `Exception` è un `Throwable`

Ogni livello introduce una specializzazione progressiva rispetto alla classe padre.

> **Significato della gerarchia**
>
> Questa organizzazione ad albero non è solo strutturale, ma ha un preciso significato progettuale:
>
> - **permette di astrarre, intercettando un'intera categoria di problemi tramite la superclasse;**
> - **consente una gestione selettiva, intercettando solo le eccezioni più specifiche;**
> - **mantiene coerenza con il principio di ereditarietà della programmazione a oggetti**.
>
> Ad esempio, **intercettando `Exception` si catturano tutte le sue sottoclassi; intercettando `IOException`, invece, si limitano le intercettazioni alle sole eccezioni appartenenti a quella specifica sotto-gerarchia.**

Classificazione delle eccezioni

Le eccezioni in Java si distinguono in due grandi categorie:

1. **Checked Exceptions**
2. **Unchecked Exceptions**

La distinzione dipende dalla posizione nella gerarchia rispetto a `RuntimeException`.

1. Eccezioni Checked

Le **checked exceptions** sono tutte le eccezioni che:

- **derivano da** `Exception`
- ****ma non derivano da** `RuntimeException` **

Sono chiamate “controllate” perché il compilatore obbliga il programmatore a:

- **intercettarle tramite** `try-catch`**, oppure**
- **dichiararle nel metodo con la clausola** `throws`**.**

Se ciò non avviene, il codice **non compila**.

Esempio:

```java
public String readFile(BufferedReader br) throws java.io.IOException{
    ...
}
```

Se si rimuove la clausola `throws IOException` (o non si gestisce l’eccezione con un `try-catch`), il compilatore segnala un errore.

Le checked exceptions rappresentano generalmente:

- **problemi esterni al programma,**
- **situazioni imprevedibili ma plausibili (file inesistenti, problemi di I/O, database non raggiungibile, ecc.).**

2. Eccezioni Unchecked

Le **unchecked exceptions** sono quelle che:

- **derivano da** `RuntimeException`**.**

Il compilatore non obbliga a dichiararle né a intercettarle.
Per questo motivo vengono chiamate “non controllate”.
Esempio:

```java
public String contaCarretteri(String testo){
    return.testo.length();
}
```

Se `testo` è `null`, durante l'esecuzione verrà sollevata una `NullPointerException`.

Il compilatore non segnala alcun errore, perché questa è una RuntimeException, quindi Unchecked.
Le unchecked exceptions:

- **sono spesso dovute a errori di programmazione,**
- **rappresentano bug logici,**
- **si manifestano solo a runtime.**

Collegamento con la gerarchia

La classificazione può essere riassunta così:

```
Throwable
 ├── Error
 └── Exception
      ├── RuntimeException  → Unchecked
      └── Altre Exception   → Checked
```

Quindi:

- **Se un'eccezione deriva da `RuntimeException` → è unchecked**
- **Se un'eccezione non deriva da `RuntimeException` → è checked**

Per chiarire:

- **Anche le unchecked possono essere intercettate con `try-catch`.**
- Semplicemente **non è obbligatorio farlo**.

La filosofia progettuale è questa:

- **Le *checked* obbligano a considerare esplicitamente certe condizioni**.
- **Le *unchecked* segnalano errori che, idealmente, dovrebbero essere evitati scrivendo codice corretto.**

In sintesi

Checked Exceptions
sono tutte le eccezioni che:

- **derivano da** `Exception`
- **ma non derivano da** `RuntimeException`

Sono dette *checked* perché il **compilatore impone** al programmatore di:

- **gestirle con un blocco** `try-catch`,
  oppure
- **dichiararle nella firma del metodo con la clausola** `throws`**.**

In caso contrario, il codice **non compila**.

Queste eccezioni rappresentano in genere:

- **condizioni esterne al programma,**
- **situazioni prevedibili ma non controllabili direttamente,**
- **eventi plausibili (file non trovato, errore di I/O, database non raggiungibile, ecc.).**

Non significa che siano sempre “esterne”, ma spesso riguardano interazioni con l’ambiente.

Unchecked Exceptions
Sono tutte le eccezioni che:

- **derivano da** `RuntimeException`**.**

Sono dette *unchecked* perché **il compilatore non obbliga a gestirle né a dichiararle.**

Rappresentano tipicamente:

- **errori logici,**
- **violazioni di precondizioni,**
- **bug di programmazione (es.** `NullPointerException`**,** `ArithmeticException`**,** `IndexOutOfBoundsException`**).**

Si manifestano esclusivamente durante l’esecuzione (runtime), quando si verifica concretamente la condizione errata.

RuntimeException

Come abbiamo già detto, le `RuntimeException` rappresentano:

- **eccezioni** non controllate (_unchecked_), **cioè errori che derivano da problemi logici nel codice.**
- **Queste eccezioni non sono obbligatorie da gestire con** `try-catch` **o da dichiarare con** `throws`**, ma si manifestano solo a runtime.**

Tipici casi di `RuntimeException` includono:

- **Cast difettoso** → **tentativo di convertire un oggetto in un tipo incompatibile**
- **Accesso a un array fuori dai limiti** → **leggere o scrivere un elemento oltre l'indice disponibile**
- **Accesso a un riferimento nullo** → **chiamare un metodo o accedere a un campo di un oggetto null**
- **Divisione per zero** → **operazione aritmetica illegale**

Esempi concreti di classi di eccezioni derivanti da `RuntimeException`:

Situazione	Classe Exception
Cast difettoso	`ClassCastException`
Accesso a un array fuori dai limiti	`ArrayIndexOutOfBoundsException`
Accesso a un riferimento nullo	`NullPointerException`
Divisione per zero	`ArithmeticException`

Queste eccezioni indicano **errori di programmazione** e, idealmente, dovrebbero essere prevenute scrivendo codice corretto.
Tuttavia, possono comunque essere gestite per rendere il programma più robusto e sicuro.

Sollevare e gestire le eccezioni in Java

Finora abbiamo visto cosa sono le eccezioni e il concetto di gestione degli errori. Ora vediamo **come sollevarle e gestirle concretamente nel codice**.

Un metodo che può generare un'eccezione deve dichiararla nella sua firma tramite lo specificatore `throws`.

Un metodo può sollevare eccezioni

Se un metodo può generare un'eccezione controllata (`checked`), **è obbligatorio indicarlo nella firma**:

Ad esempio:

```java
public String readLine() throws IOException {
    ...
}
```

- Qui il metodo `readLine` **dichiara che può generare un'eccezione di tipo** `IOException` **.**
- **Qualsiasi codice che chiama questo metodo deve gestire l'eccezione con un** `try-catch` **oppure dichiararla a sua volta con** `throws` **.**

Se il metodo può sollevare **più eccezioni**, tutte devono essere indicate nella firma, separate da una virgola:

```java
public void apriFile() throws EOFException, MalformedURLException {
    ...
}
```

> Nota:
> **solo le eccezioni controllate (** `checked` **) devono essere dichiarate.**
> **Le eccezioni non controllate (** `unchecked` **, come** `RuntimeException` **) possono essere sollevate senza specificarle con** `throws` **.**

Infine, un metodo deve sempre dichiarare:

- **Il tipo di ritorno**, se presente;
- **Tutte le eccezioni checked** che potrebbe lanciare.

Lanciare un'eccezione (`throw`)

Quando un metodo rileva una condizione anomala, **può sollevare (lanciare) un'eccezione per interrompere il flusso normale e trasferire il controllo a un gestore di errori.**

I passaggi principali sono:

1. **Individuare la classe di eccezione più appropriata**:
   Scegliere il tipo di eccezione che meglio descrive l'anomalia riscontrata (ad esempio `EOFException` per fine file).
2. **Creare un'istanza dell'eccezione**:
   Si utilizza il costruttore della classe di eccezione.
3. **Lanciare l'eccezione** con la keyword `throw`:

Esempio pratico:

Java

```java
String readData(BufferedReader in) throws EOFException {
    ...
    while (...) {
        if (ch == -1) {  // raggiunto la fine del file
            if (n < len) {
                throw new EOFException(); // crea e lancia l'eccezione
            }
        }
        ...
    }
    return s;
}
```

- **La dichiarazione `throws EOFException` nella firma del metodo indica che questo metodo può generare un'eccezione di tipo `EOFException`.**
- Il comando `throw new EOFException();` **crea un oggetto eccezione e lo lancia, interrompendo il flusso normale del metodo.**

> In sintesi: **per generare un'eccezione è necessario dichiararla, crearla e rilanciarla nel punto del codice in cui si verifica l'anomalia.**

Chi lancia e chi gestisce un'eccezione

Quando un metodo in Java lancia un'eccezione (`throw`):

1. **Il metodo lancia l'eccezione**

- **Interrompe il flusso normale del programma.**

- **Segnala che è avvenuto un problema.**

2. **Il chiamante deve gestire l'eccezione**

- **Oppure intercettarla subito con un** `try-catch`
- **Oppure dichiararla a sua volta nella firma con** `throws` **per passarla a chi chiama quel metodo.**

Nota: questo obbligo vale solo per le **eccezioni checked**. Le **unchecked** **(** `RuntimeException` **)** possono essere lanciate senza dichiararle, ma il programmatore può comunque gestirle se lo desidera.
Esempio integrativo:

Java

```java
// Metodo che lancia un'eccezione checked
public void leggiFile() throws IOException {
  if (fileNonTrovato) {
      throw new IOException("File non trovato");
  }
}

// Chiamante che gestisce l'eccezione
try {
   leggiFile();
} catch (IOException e) {
  System.out.println("Gestita eccezione: " + e.getMessage());
}

```

Eccezioni personalizzate in Java

Oltre alle eccezioni predefinite, è possibile **creare eccezioni personalizzate** per modellare errori specifici della propria applicazione.

- **Una eccezione personalizzata deve estendere** `Exception` **o una sua sottoclasse.**
- **È buona pratica dare un nome significativo, che descriva il tipo di anomalia o errore gestito.**

Esempio di definizione:

```java
public class MyException extends Exception {

    // Costruttore di default
    public MyException() {
        super();
    }

    // Costruttore con messaggio
    public MyException(String reason) {
        super(reason);
    }
}
```

`MyException` è ora una **classe di eccezione completamente valida**, che può essere lanciata e gestita come qualsiasi altra eccezione.

Utilizzo di eccezioni personalizzate

Per lanciare la tua eccezione personalizzata:

```java
public String myMethod(...) throws MyException {
    String s = "";
    ...
    if (/* rilevata una condizione di errore */) {
        throw new MyException();  // solleva l'eccezione personalizzata
    }
    ...
    return s;  // ritorna un valore normale se non ci sono errori
}
```

- **La firma del metodo dichiara `throws MyException`, obbligando chi chiama il metodo a gestire l'eccezione (perché è checked).**
- **L'uso di `throw` e `return` ha effetti simili in termini di flusso del programma (entrambi interrompono il flusso in quel punto), ma non vanno mai usati insieme per lo stesso ramo logico.**

> In sintesi: **le eccezioni personalizzate ti permettono di modellare errori specifici della tua applicazione, rendendo il codice più chiaro e coerente**

> **con la logica di business.**

Propagazione delle eccezioni

In Java, quando un metodo **chiama un altro metodo che può sollevare un'eccezione**, ci sono due possibili comportamenti:

1. **Rilanciare l'eccezione**
   - **Si dichiara l'eccezione nella firma del metodo usando `throws`.**
   - **L'eccezione si propaga al metodo chiamante, che può a sua volta rilanciarla o gestirla.**
   - **Questo meccanismo permette di delegare la gestione dell'errore al metodo più appropriato, in base al ruolo della classe nel programma.**
2. **Gestire direttamente l'eccezione**
   - **Il metodo intercetta l'eccezione con un blocco `try-catch` e prende le opportune contromisure.**
   - **Il flusso del programma non si interrompe, a meno che il blocco catch non rilanci l'eccezione.**

Se **nessun metodo** intercetta l'eccezione, nemmeno il `main`, allora la **JVM termina l'esecuzione** del programma e stampa lo **stacktrace**, mostrando l'intera catena di metodi che ha portato all'eccezione.

> In sintesi: **la propagazione permette di trasferire la responsabilità della gestione dell'errore al metodo più appropriato, senza interrompere subito l'esecuzione del programma.**

Esempio di rilancio di un'eccezione

Supponiamo di avere due metodi, `metodo1()` e `metodo2()`, dove `metodo2()` può sollevare un'eccezione di tipo `TipoException`.

Per **rilanciare l'eccezione**, si utilizza la parola chiave `throws` nella firma del metodo:

Java

```java
void metodo2() throws TipoException {
    // codice che può generare TipoException
}

```

```java
int metodo1() throws TipoException {
    // metodo1 chiama metodo2
    metodo2();
    // eventuale altra logica
    return 0;
}

```

- `metodo2()` **crea l'eccezione**: **individua la condizione anomala e la lancia con** `throw`**.**
- `metodo1()` **riceve l'eccezione** e, **dichiarando** `throws TipoException`**, la rilancia al suo chiamante senza gestirla direttamente.**

> In pratica, **l'eccezione "sale" lungo la catena dei metodi fino a quando non viene intercettata da un blocco** `try-catch`**. Se nessuno la gestisce, il programma termina mostrando lo stacktrace.**

Intercettare le eccezioni con `try-catch`

Quando un metodo **decide di gestire direttamente un'eccezione** invece di rilanciarla, si utilizza il **blocco** `try-catch`.

- **In questo caso non è più necessario dichiarare l'eccezione nella firma con** `throws`**.**
- **Il blocco** `try` **contiene il codice che potenzialmente può generare un'eccezione.**
- **Il blocco** `catch` **intercetta l'eccezione e contiene il codice per gestirla.**

Sintassi di base

Java

```java
try {
    // codice dove si potrebbe verificare un errore
} catch (ExceptionType exc) {
    // codice per gestire l'eccezione
}
```

Se all'interno del blocco `try` viene generata un'eccezione:

1. **Il resto del codice del blocco** `try` **viene saltato.**

2. **Viene eseguito il blocco `catch`, dove si gestisce l'eccezione.**
   Esempio pratico

Java

```java
void metodo2() throws TipoException {
    // codice che può generare TipoException
}

int metodo1() {
    try {
        metodo2(); // chiamata che potrebbe sollevare eccezione
    } catch (TipoException exc) {
        // gestione dell'eccezione
        System.out.println("Errore gestito: " + exc.getMessage());
    }
    // il flusso continua normalmente
    return 0;
}
```

- `metodo2()` **genera l'eccezione.**
- `metodo1()` **la intercetta con `catch` e gestisce l'anomalia senza interrompere l'esecuzione del programma.**

> In sintesi: **il blocco `try-catch` consente di intercettare e gestire le eccezioni, evitando che l'applicazione termini inaspettatamente.**

Intercettare più eccezioni

In un singolo blocco `try` è possibile gestire più tipi di eccezioni, ciascuna con un proprio blocco `catch`.

- **Ordine importante: le eccezioni più specifiche devono essere intercettate prima, seguite da quelle più generiche.**
- **Questo evita che una eccezione specifica venga catturata dal blocco di una superclasse, impedendo la gestione dedicata.**

Sintassi

Java

```java
try {
```

```
    // codice che potrebbe generare eccezioni diverse
}
catch (MalformedURLException e1) {
    // gestione URL non valido
}
catch (UnknownHostException e2) {
    // gestione host non trovato
}
catch (IOException e3) {
    // gestione errore generico di I/O
}
```

- `MalformedURLException` **e** `UnknownHostException` **sono eccezioni specifiche, figlie di** `IOException`**.**
- `IOException` **è più generica e intercetta qualsiasi altra eccezione di input/output non gestita dai catch precedenti.**

> In sintesi: **utilizzare più blocchi** `catch` **permette di gestire ciascuna eccezione in modo mirato, mantenendo il codice chiaro e robusto.**

Gestione di più eccezioni con `OR` (Java 7+)

A partire da Java 7, è possibile **intercettare più eccezioni nello stesso blocco** `catch`, usando il simbolo `|` per indicare una relazione **OR** tra le eccezioni.

Esempio

Java

```
try {
    // codice che può generare diverse eccezioni
}
catch (MalformedURLException | UnknownHostException e) {
    // gestione URL non valido oppure host ignoto
}
catch (IOException e3) {
    // gestione errore generico di I/O
}
```

- **Il primo** `catch` **intercetta sia** `MalformedURLException` **sia** `UnknownHostException`**, eseguendo lo stesso blocco di codice per entrambe**.

- **Il secondo `catch` intercetta eventuali altre eccezioni di tipo `IOException` non intercettate dal blocco precedente.**
- Come sempre, **è importante gestire prima le eccezioni più specifiche, poi quelle più generiche.**

> **Questa sintassi riduce la ridondanza del codice e rende più leggibile la gestione di eccezioni simili.**

Meccanismo vincolante delle eccezioni

In Java, la gestione delle eccezioni non è soltanto una questione di sintassi:

- esiste un **meccanismo vincolante** che garantisce la robustezza e la prevedibilità del codice.
  Questo meccanismo si basa su due aspetti principali:

1. **gestione diretta**
2. **propagazione** delle eccezioni.

1. Gestione diretta con `try-catch`

Quando un metodo potrebbe generare un'eccezione, **possiamo intercettarla direttamente con un blocco `try-catch`.**

- **Il codice che potrebbe causare l'errore viene inserito nel blocco `try`.**
- Se l'eccezione si verifica, **l'esecuzione salta al blocco `catch` corrispondente, che contiene le istruzioni per gestire la situazione.**
- Questo permette di **prevenire l'interruzione del programma** e di reagire in modo controllato a situazioni impreviste, come errori di input, file non trovati o divisioni per zero.

Esempio:

Java

```java
try {
    int risultato = divisore / divisore2;
} catch (ArithmeticException e) {
    System.out.println("Divisione per zero!");
}
```

Qui, l'eccezione viene intercettata subito, evitando che il programma termini in modo anomalo.

2. Propagazione con `throws`

In alternativa, **un metodo può rilanciare l'eccezione al chiamante usando la keyword `throws` nella firma del metodo.**

- In questo caso, **il metodo non gestisce direttamente l'errore, ma trasferisce la responsabilità a chi lo invoca.**
- **La propagazione è utile quando il metodo chiamante è più adatto a capire come gestire l'eccezione, secondo il contesto applicativo.**

Esempio:

Java

```java
void metodo2() throws IOException {
    // codice che potrebbe generare IOException
}

void metodo1() throws IOException {
    metodo2();  // rilancio l'eccezione al chiamante
}
```

Rilancio delle eccezioni dopo l'intercettazione

In Java, può capitare di voler **gestire un'eccezione parzialmente**, ad esempio per eseguire del logging o liberare risorse, **ma allo stesso tempo avvisare il metodo chiamante che si è verificato un problema**.

Questo si realizza **rilanciando l'eccezione all'interno del blocco `catch`**.

Struttura generale:

Java

```java
try {
    // codice che potrebbe generare un'eccezione
} catch (TipoException exc) {
    // gestiamo l'eccezione localmente
    metodoDiGestione();

    // rilanciamo l'eccezione per informare il chiamante
```

```
        throw exc;
    }
```

- **Quando il codice all'interno del `try` genera un'eccezione, l'oggetto dell'eccezione viene passato al blocco `catch` come `exc`.**
- All'interno del `catch`, **possiamo eseguire operazioni locali come stampare un messaggio di errore, fare il logging, o liberare risorse.**
- Infine, con `throw exc;` **l'eccezione viene rilanciata al metodo chiamante, che potrà decidere se gestirla ulteriormente o propagare a sua volta.**
  Esempio pratico:

Java

```java
void metodoA() throws IOException {
    try {
        metodoB();  // può generare IOException
    } catch (IOException e) {
        System.out.println("Errore in metodoA: log del problema");
        // rilancio l'eccezione per non interrompere il flusso previsto dal chiamante
        throw e;
    }
}

void metodoB() throws IOException {
    throw new IOException("File non trovato");
}
```

Risultato:

- `metodoA()` **intercetta l'eccezione per fare il logging,**
- **ma l'eccezione continua a propagarsi fino al chiamante di `metodoA()`.**

> In sintesi: **rilanciare un'eccezione dal `catch` consente di combinare gestione locale e propagazione allo stesso tempo.**

Il blocco `finally` in Java

In Java, oltre a `try` e `catch`, esiste il **blocco `finally`:**

- **che permette di eseguire del codice sempre, indipendentemente dal**

fatto che si sia verificata un'eccezione o meno.

Struttura generale:

Java

```java
try {
    // codice che potrebbe generare un'eccezione
} catch (Exception e) {
    // gestione dell'eccezione
} finally {
    // codice che viene eseguito sempre
}
```

- Il codice all'interno di `finally` **viene eseguito in ogni caso**, anche se:
  1. **Non viene sollevata alcuna eccezione;**
  2. **Si verifica un'eccezione e viene intercettata dal** `catch`**;**
  3. **Si verifica un'eccezione non intercettata, come una** `RuntimeException` **non gestita.**
- È molto utile per **rilasciare risorse di sistema**, come:
  - **chiudere file,**
  - **liberare connessioni di rete o database,**
  - **liberare memoria temporanea.**

Interazione con `break`, `continue` e `return`

- Anche se nel `try` o nel `catch` sono presenti istruzioni `break`, `continue` o `return`, **il blocco** `finally` **viene comunque eseguito prima** che il flusso venga effettivamente interrotto o restituito.
  Esempio pratico:

Java

```java
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader("file.txt"));
    String line = reader.readLine();
    System.out.println(line);
} catch (IOException e) {
    System.out.println("Errore durante la lettura del file");
} finally {
    // chiusura del file garantita
```

```java
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            System.out.println("Errore durante la chiusura del file");
        }
    }
}
```

In sintesi: **il blocco `finally` garantisce che il codice critico venga sempre eseguito, assicurando il corretto rilascio di risorse e la coerenza del programma anche in caso di eccezioni.**